République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Sousse

Institut Supérieur des Sciences
Appliquées et de Technologie
de Sousse

DEPARTEMENT INFORMATIQUE

RAPPORT DE MINI PROJET WEB

FIA3

Quiz Web :
Page de Quiz Interactif

Elaboré par :

Étudiant 1 : IBRAHIM Mohamed Salah

Étudiant 2 : BELGUITH Tesnim

Année Universitaire : 2026/2027

Plan :

1 Introduction **3**

2 Cahier des charges / Objectifs **3**

3 Conception **4**
- 3.1 Structure du dossier . . . 4
- 3.2 Architecture générale . . . 4
- 3.3 Structure HTML . . . 4
- 3.4 Charte graphique . . . 5
- 3.5 Outils utilisés . . . 5

4 Implémentation (Code et explications) **5**
- 4.1 Exemple de code HTML . . . 5
- 4.2 Exemple de code CSS . . . 8
- 4.3 Exemple de code JavaScript . . . 14
- 4.4 Exemple de code serveur (Node.js) . . . 19

5 Résultat final **22**
- 5.1 Captures d'écran . . . 22
- 5.2 Commentaire du rendu . . . 24
- 5.3 Performance . . . 24
- 5.4 Validation W3C . . . 25
- 5.5 Vidéo de démonstration . . . 26

6 Difficultés rencontrées et solutions **27**

7 Conclusion **27**

1 Introduction

Le développement web repose sur trois technologies fondamentales : HTML pour la structure, CSS pour la présentation et JavaScript pour l'interactivité. Dans le cadre de ce mini projet web, nous avons réalisé une **page de quiz interactif** intitulée « Quiz Web », qui permet à l'utilisateur de tester ses connaissances, d'abord en HTML, CSS et JavaScript grâce à un quiz de démonstration, puis sur **n'importe quel sujet** grâce à la génération automatique de questions par l'API Gemini.

L'application se compose d'un écran d'accueil (réglages du quiz et meilleurs scores), d'un écran de questions à choix multiples ou vrai/faux avec compte à rebours, correction immédiate et explication, puis d'un écran de résultat affichant le score en points. Un petit serveur Node.js sert les fichiers de la page et appelle l'API Gemini à la place du navigateur, afin que la clé d'API ne soit jamais exposée au visiteur. Ce rapport présente le cahier des charges, la conception, l'implémentation, le résultat final ainsi que les difficultés rencontrées.

2 Cahier des charges / Objectifs

L'objectif de ce projet est de concevoir une page web interactive, moderne et accessible. La partie interface reste en HTML, CSS et JavaScript pur, sans framework ni bibliothèque externe (hormis les polices Google Fonts) ; seul le serveur local, écrit en Node.js sans dépendance npm, est ajouté pour la génération des questions. Le quiz doit guider l'utilisateur de manière fluide, depuis l'écran d'accueil jusqu'à l'affichage du score final.

Les fonctionnalités attendues sont les suivantes :

— un **écran d'accueil** présentant le quiz, un formulaire de réglages (sujet, nombre de questions, type, niveau), deux boutons (« Générer mon quiz » et « Jouer au quiz démo ») et la liste des meilleurs scores ;
— la **génération des questions par l'API Gemini** à partir d'un sujet libre, de 3 à 10 questions, au format QCM (quatre choix), vrai/faux ou mixte, avec trois niveaux de difficulté ;
— un **quiz de démonstration** de cinq questions (HTML, CSS, JavaScript) qui fonctionne sans clé d'API ni connexion Internet ;
— une **barre de progression** et un compteur de questions ;
— un **compte à rebours de 20 secondes** par question, avec une barre de temps qui passe au rouge dans les cinq dernières secondes ;
— un **score en points** : une bonne réponse rapporte jusqu'à 100 points, diminués de 5 points par seconde écoulée (minimum 10 points) ; une mauvaise réponse ou une réponse hors délai vaut zéro ;
— un **retour immédiat** (réponse correcte, incorrecte ou temps écoulé) accompagné d'une explication pédagogique ;
— un **écran de résultat** avec le score, le nombre de bonnes réponses, un message adapté, les meilleurs scores et deux boutons : « Rejouer » (même série) et « Nouveau quiz » (retour à l'accueil) ;
— la **sauvegarde des cinq meilleurs scores** dans le stockage local du navigateur ;
— un **design responsive**, adapté aux ordinateurs, tablettes et téléphones ;
— le respect des bonnes pratiques d'**accessibilité** (gestion du focus, navigation au clavier, annonces vocales, réduction des animations) ;
— la **sécurité de la clé d'API** : elle reste côté serveur, dans un fichier `.env` ignoré par Git.

3 Conception

3.1 Structure du dossier

```
/Quiz_Mini_Projet
|-- index.html      (structure de la page)
|-- style.css       (mise en forme)
|-- script.js       (logique du quiz, côté navigateur)
|-- server.js       (serveur Node.js + appel à l'API Gemini)
|-- package.json    (script "npm start", Node.js >= 18)
|-- .env.example    (modèle de configuration)
|-- .env            (clé GEMINI_API_KEY, ignoré par Git)
|-- .gitignore
|-- README.md
```

3.2 Architecture générale

Le projet suit une architecture client/serveur minimale. Le navigateur affiche la page et exécute le quiz ; il ne contacte jamais directement l'API Gemini. Lorsque l'utilisateur demande un quiz sur un sujet, `script.js` envoie une requête `POST /api/questions` au serveur local. Celui-ci valide les paramètres, ajoute la clé d'API (lue dans `.env`), interroge Gemini, contrôle la réponse puis renvoie au navigateur une liste de questions au format JSON.

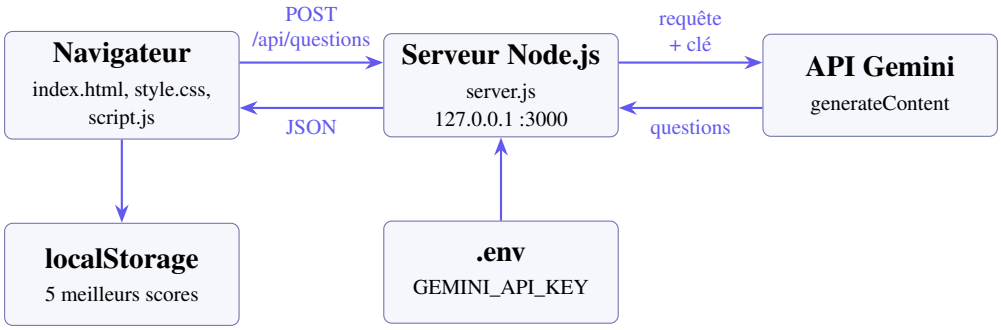


FIGURE 1 – Architecture de l'application

Le serveur n'expose que quatre fichiers (`/`, `/index.html`, `/style.css`, `/script.js`) et une route d'API ; toute autre adresse renvoie une erreur 404. Il écoute uniquement sur `127.0.0.1`, c'est-à-dire en usage local.

3.3 Structure HTML

La page est organisée de manière logique autour d'un élément `<main class="app-shell">` qui contient :

- un `<header>` (barre supérieure avec le logo « quizweb ») ;
- trois `<section>` correspondant aux trois écrans de l'application : accueil (`#accueil`), quiz (`#quiz`) et résultat (`#result`) ;
- un `<footer>` (pied de page).

Un seul écran est visible à la fois : les deux autres reçoivent la classe `hidden` (`display: none`).

3.4 Charte graphique

— **Couleurs** : violet principal `#6259ef` (boutons, accents), lavande `#f0efff` (fonds secondaires), vert `#219b72` pour les bonnes réponses et rouge `#d9536c` pour les mauvaises et le temps presque écoulé, sur un fond clair `#f7f8fc` ;
— **Polices** : *Manrope* pour les titres et *DM Sans* pour le texte courant (Google Fonts) ;
— **Thème visuel** : interface épurée, cartes aux coins arrondis, ombres douces, petites animations (flottement, apparition progressive des questions) et symboles décoratifs.

3.5 Outils utilisés

— **VS Code** comme éditeur de code ;
— **Node.js** (version 18 ou plus) pour le serveur local, lancé avec `npm start` ;
— l'**API Gemini** (`generateContent`, modèle par défaut `gemini-3.5-flash-lite`) pour la génération des questions ;
— un **navigateur web** et ses outils de développement (console, mode responsive) ;
— **Lighthouse** et le **validateur W3C** pour contrôler la qualité du code ;
— **Google Fonts** pour les polices.

4 Implémentation (Code et explications)

4.1 Exemple de code HTML

L'écran d'accueil est un élément `<section>` relié à son titre par l'attribut `aria-labelledby`, ce qui facilite la lecture par les lecteurs d'écran :

```
<section class="screen welcome-screen" id="accueil"
         aria-labelledby="welcome-title">
  <div class="welcome-copy">
    <h1 id="welcome-title">Le web, vous connaissez
      <span class="title-accent">vraiment ?</span></h1>
    <p class="welcome-description">Quelques questions, un peu
      de réflexion et beaucoup de fun.</p>
    ...
  </div>
</section>
```

Balise `<head>` (extrait simplifié). Elle déclare l'encodage, l'affichage adapté aux mobiles (`viewport`), la description de la page, le chargement des polices Google Fonts, la feuille de style et le script. L'attribut `defer` permet d'exécuter le JavaScript après l'analyse du HTML :

```
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#f7f8fc">
  <meta name="description" content="Un petit quiz de culture
        web pour tester vos connaissances en HTML, CSS et
        JavaScript.">
  <title>Quiz Web - À vous de jouer !</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
```

```
  <link href="https://fonts.googleapis.com/css2?family=DM+Sans
        &family=Manrope&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
```

En-tête (header). La barre supérieure contient le logo cliquable. L'attribut `aria-label` donne un nom accessible au lien, et `aria-hidden="true"` masque aux lecteurs d'écran les éléments purement décoratifs :

```
<header class="topbar">
  <a class="brand" href="#accueil" aria-label="Quiz Web, accueil">
    <span class="brand-mark" aria-hidden="true">q.</span>
    <span>quiz<span class="brand-light">web</span></span>
  </a>
</header>
```

Formulaire de réglages. Le bloc de l'écran d'accueil permet de choisir le sujet, le nombre de questions, le type et le niveau. Chaque champ est associé à une étiquette `<label for="...">`. Deux boutons lancent soit la génération par l'IA, soit le quiz de démonstration ; le message d'état est une zone `role="status"` annoncée par les lecteurs d'écran (extrait simplifié) :

```
<div class="quiz-settings" aria-label="Personnaliser le quiz">
  <label for="topic-input">Sujet du quiz</label>
  <input id="topic-input" type="text" maxlength="80"
         value="Culture web">
  <div class="settings-row">
    <div><label for="question-count">Questions</label>
      <select id="question-count">
        <option value="5" selected>5</option>
        <option value="3">3</option> ...
      </select></div>
    <div><label for="question-type">Type</label> ...</div>
    <div><label for="difficulty">Niveau</label> ...</div>
  </div>
  <button class="button button-primary start-button"
          id="generate-button" type="button">Générer mon quiz
    <span aria-hidden="true">...</span></button>
  <button class="button button-secondary demo-button"
          id="demo-button" type="button">Jouer au quiz démo
  </button>
  <p class="setup-message hidden" id="setup-message"
     role="status" aria-live="polite"></p>
</div>
```

Meilleurs scores. La liste ordonnée est volontairement presque vide : son contenu est généré par JavaScript à partir du `localStorage`.

```
<div class="high-scores" aria-labelledby="high-scores-title">
  <h2 id="high-scores-title">Meilleurs scores</h2>
```

```
  <ol id="high-scores-list">
    <li>Vos résultats apparaîtront ici.</li>
  </ol>
</div>
```

Écran du quiz. Cette section regroupe le compteur, la barre de progression, le minuteur et la carte de question. Elle est masquée au départ grâce à la classe `hidden`. Les deux barres utilisent le rôle `progressbar` avec ses attributs `aria-value*` :

```
<section class="screen quiz-screen hidden" id="quiz"
         aria-labelledby="question-title">
  <div class="quiz-counter">
    <span id="question-number">01</span>
    <span class="counter-slash">/</span>
    <span id="question-total">05</span>
  </div>
  <div class="progress-track" role="progressbar"
       aria-label="Progression du quiz"
       aria-valuemin="0" aria-valuemax="5" aria-valuenow="1">
    <div class="progress-fill" id="progress-fill"></div>
  </div>
  <div class="timer-row">
    <span id="question-kind" class="question-kind">QCM</span>
    <div class="timer-wrap">
      <span class="timer-label">Temps restant</span>
      <strong id="timer-countdown" aria-live="off">20 s</strong>
    </div>
  </div>
  <div class="timer-track" id="timer-track" role="progressbar"
       aria-label="Temps restant pour répondre"
       aria-valuemin="0" aria-valuemax="20" aria-valuenow="20">
    <div class="timer-fill" id="timer-fill"></div>
  </div>
  ...
```

Carte de question. La zone de réponses est volontairement vide dans le HTML : les boutons sont générés dynamiquement par JavaScript pour chaque question. Le titre porte `tabindex="-1"` pour pouvoir recevoir le focus par programmation :

```
<div class="question-card" id="question-card" aria-live="polite">
  <span class="question-category" id="question-category">HTML</span>
  <h1 id="question-title" tabindex="-1">Chargement...</h1>
  <div class="answers" id="answers" role="group"
       aria-label="Choix de réponse"></div>
  <div class="feedback hidden" id="feedback"
       role="status" aria-live="polite"></div>
  <button class="button button-primary next-button"
          id="next-button" type="button" disabled>Suivant</button>
</div>
```

Écran de résultat. Le titre, le message, le score et la barre sont des éléments dont le contenu est rempli par JavaScript. Deux boutons permettent de rejouer la même série ou de revenir à

l'accueil :

```
<section class="screen result-screen hidden" id="result"
         aria-labelledby="result-title">
  <div class="result-card">
    <p class="eyebrow">QUIZ TERMINÉ</p>
    <h1 id="result-title">Bien joué !</h1>
    <p class="result-message" id="result-message"></p>
    <div class="score-display">
      <strong id="score-value">0</strong><span>pts</span>
    </div>
    <p class="score-caption" id="score-caption"></p>
    <div class="result-progress" aria-hidden="true">
      <div class="result-progress-fill"
           id="result-progress-fill"></div></div>
    <div class="result-actions">
      <button class="button button-primary restart-button"
              id="restart-button" type="button">Rejouer</button>
      <button class="button button-secondary home-button"
              id="home-button" type="button">Nouveau quiz</button>
    </div>
    <div class="result-high-scores">
      <h2>Meilleurs scores</h2>
      <ol id="result-high-scores-list"></ol>
    </div>
  </div>
</section>
```

Pied de page (footer).

```
<footer class="site-footer">
  <span>© 2026 quizweb</span>
  <span>Fait pour les esprits curieux</span>
</footer>
```

4.2 Exemple de code CSS

Les couleurs et la police sont définies une seule fois dans des variables CSS (`:root`), puis réutilisées dans toute la feuille de style :

```
:root {
  --ink: #20223b;
  --purple: #6259ef;
  --lavender: #f0efff;
  --green: #219b72;
  --red: #d9536c;
  --surface: #fff;
  font-family: "DM Sans", sans-serif;
}

.app-shell {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  max-width: 1440px;
```

```
  margin: 0 auto;
}

.welcome-screen {
  display: grid;
  grid-template-columns: 1.04fr .96fr;
  align-items: center;
}
```

En-tête et logo. Flexbox aligne verticalement le contenu de la barre supérieure, dont la hauteur est fixe, et une ligne de séparation la distingue du reste de la page :

```
.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 92px;
  border-bottom: 1px solid #ebebf2;
}

.brand-mark {
  display: grid;
  width: 34px;
  height: 34px;
  place-items: center;
  border-radius: 11px;
  background: var(--purple);
  color: white;
}
```

Boutons et effet de survol. Le bouton principal se soulève légèrement au survol grâce à `transform` et `transition` ; le bouton secondaire (« Jouer au quiz démo », « Nouveau quiz ») est blanc avec une bordure claire :

```
.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 49px;
  padding: 0 23px;
  border: 0;
  border-radius: 13px;
  cursor: pointer;
  transition: transform .2s ease, background .2s ease,
              box-shadow .2s ease;
}

.button:hover:not(:disabled) { transform: translateY(-2px); }

.button-primary {
  color: white;
  background: var(--purple);
  box-shadow: 0 7px 17px #6259ef35;
}
```

```
.button-secondary {
  border: 1px solid #e4e3ef;
  background: #fff;
  color: #5e5f78;
}

.button:disabled { opacity: .65; cursor: wait; }
```

Formulaire de réglages. Les trois listes déroulantes (nombre, type, niveau) sont placées sur une même ligne grâce à une grille de trois colonnes égales. Le focus clavier est mis en évidence par un contour :

```
.quiz-settings { display: grid; gap: 9px; max-width: 430px; }

.settings-row {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 10px;
}

.quiz-settings input, .quiz-settings select {
  width: 100%;
  height: 40px;
  border: 1px solid #e5e5ee;
  border-radius: 9px;
}

.quiz-settings input:focus, .quiz-settings select:focus {
  outline: 3px solid #a9a3ff66;
  border-color: var(--purple);
}

.setup-message.setup-error { background: #fff2f4; color: #a44358; }
```

Barre de progression. La largeur de la barre colorée est modifiée par JavaScript ; la transition rend l'avancement fluide :

```
.progress-track {
  height: 5px;
  overflow: hidden;
  border-radius: 9px;
  background: #e9e9f2;
}

.progress-fill {
  width: 20%;
  height: 100%;
  border-radius: inherit;
  background: linear-gradient(90deg, #635af0, #9b8cff);
  transition: width .45s ease;
}
```

Minuteur. La barre de temps se vide de façon linéaire et devient rouge lorsque la classe `timer-low` est ajoutée par JavaScript (cinq dernières secondes) :

```
.timer-track {
  height: 4px;
  overflow: hidden;
  border-radius: 9px;
  background: #ececf3;
}

.timer-fill {
  width: 100%;
  height: 100%;
  background: #70c9a4;
  transition: width .2s linear, background .2s ease;
}

.timer-track.timer-low .timer-fill { background: var(--red); }
.timer-wrap strong.timer-text-low  { color: var(--red); }
```

Carte de question et options de réponse. Les réponses sont disposées avec `display: grid` et changent d'aspect selon leur état (survol, correcte, incorrecte). Une animation d'arrivée est rejouée à chaque nouvelle question :

```
.question-card {
  padding: 32px 37px 25px;
  border: 1px solid #ececf3;
  border-radius: 19px;
  background: var(--surface);
  box-shadow: 0 16px 45px #31304f0c;
}

.answers { display: grid; gap: 10px; }

.answer-option {
  display: flex;
  align-items: center;
  width: 100%;
  min-height: 53px;
  padding: 0 15px;
  border: 1px solid #e9e9f0;
  border-radius: 11px;
  background: white;
  cursor: pointer;
  transition: border .18s ease, background .18s ease,
              transform .18s ease;
}

.answer-option:hover:not(:disabled) {
  transform: translateX(3px);
  border-color: #b9b5ff;
}

.answer-option.correct {
  border-color: #7bd4b2;
  background: #effaf5;
```

```
  color: #25795d;
}

.answer-option.incorrect {
  border-color: #efa0ae;
  background: #fff3f5;
  color: #aa4257;
}

.question-card.question-enter { animation: question-arrive .28s ease both; }
```

Titre adaptable. La fonction `clamp()` fixe une taille de police minimale, une taille idéale proportionnelle à la largeur de l'écran et une taille maximale :

```
.welcome-copy h1 {
  font-size: clamp(42px, 5vw, 66px);
  line-height: 1.11;
}
```

Accessibilité : focus visible. Un contour net apparaît sur les boutons et les liens lorsque l'utilisateur navigue au clavier :

```
button:focus-visible, a:focus-visible {
  outline: 3px solid #a9a3ff;
  outline-offset: 4px;
}
```

Animations. Une animation d'apparition (`fade-up`) est appliquée à chaque nouvel écran et un effet de flottement à la carte de la page d'accueil :

```
.quiz-screen { animation: fade-up .4s ease both; }

@keyframes fade-up {
  from { opacity: 0; transform: translateY(9px); }
  to   { opacity: 1; transform: translateY(0); }
}

@keyframes float {
  0%, 100% { transform: translateY(0) rotate(-3deg); }
  50%      { transform: translateY(-9px) rotate(-2deg); }
}
```

Responsive design (media queries). Sur écran étroit, la page d'accueil passe sur une seule colonne, les marges sont réduites et les listes déroulantes se resserrent :

```
@media (max-width: 760px) {
  .app-shell { padding: 0 6%; }
  .welcome-screen { grid-template-columns: 1fr; }
  .question-card { padding: 26px 22px 20px; }
```

```
}

@media (max-width: 440px) {
  .welcome-copy h1 { font-size: 43px; }
  .settings-row { gap: 7px; }
  .answer-text { font-size: 12px; }
}
```

Accessibilité : réduction des animations. Les utilisateurs qui ont désactivé les animations dans leur système ne les voient pas :

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: .01ms !important;
    transition-duration: .01ms !important;
  }
}
```

Retour sur la réponse. Le message d'explication prend un fond vert ou rouge selon que la réponse est correcte ou non :

```
.feedback.correct-feedback   { background: #effaf5; color: #24795d; }
.feedback.incorrect-feedback { background: #fff3f5; color: #9b4557; }
```

Écran de résultat. La carte du score est centrée, la barre de résultat se remplit grâce à une transition sur sa largeur, et les deux boutons d'action passent à la ligne si la place manque :

```
.result-card {
  width: min(100%, 390px);
  padding: 35px 38px 32px;
  border-radius: 20px;
  background: white;
  text-align: center;
}

.result-actions {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 9px;
}

.result-progress-fill {
  width: 0;
  height: 100%;
  background: linear-gradient(90deg, #635af0, #9b8cff);
  transition: width .8s ease;
}
```

Masquage des écrans. Un seul écran est visible à la fois : la classe utilitaire `hidden` masque les autres.

```
.hidden { display: none !important; }
```

4.3 Exemple de code JavaScript

Le script commence par les constantes du jeu : durée d'une question, points maximum, pénalité par seconde et clé de stockage des meilleurs scores. Le quiz de démonstration est un tableau d'objets contenant la catégorie, le type, l'énoncé, les réponses, l'indice de la bonne réponse et une explication. Les questions générées par l'IA ont exactement le même format.

```
const QUESTION_TIME = 20;
const POINTS_PER_QUESTION = 100;
const TIME_PENALTY_PER_SECOND = 5;
const HIGH_SCORES_KEY = "quizweb-high-scores-v1";

const demoQuestions = [
  {
    category: "HTML",
    type: "multiple_choice",
    question: "Quelle balise HTML sert à créer un lien vers une autre page
   ?",
    answers: ["<link>", "<a>", "<href>", "<url>"],
    correct: 1,
    explanation: "La balise <a> (pour « anchor ») crée un lien. ..."
  },
  // ... quatre autres questions (QCM et vrai/faux)
];
```

Variables et éléments de la page. Le script mémorise l'état du quiz dans quelques variables, et récupère les éléments HTML dont il a besoin avec `getElementById` :

```
let currentQuestions = [];    // série de questions en cours
let currentQuestion = 0;      // indice de la question affichée
let score = 0;                // points cumulés
let correctCount = 0;         // nombre de bonnes réponses
let hasAnswered = false;      // la question a-t-elle reçu une réponse ?
let timerId = null;           // identifiant du minuteur
let questionStartedAt = 0;    // heure d'affichage de la question

const generateButton = document.getElementById("generate-button");
const demoButton = document.getElementById("demo-button");
const nextButton = document.getElementById("next-button");
const progressFill = document.getElementById("progress-fill");
const answersContainer = document.getElementById("answers");
```

Affichage des écrans. La fonction `showScreen` ajoute la classe `hidden` à tous les écrans, puis la retire de celui à afficher :

```
function showScreen(screenToShow) {
```

```
  [welcomeScreen, quizScreen, resultScreen]
    .forEach((screen) => screen.classList.add("hidden"));
  screenToShow.classList.remove("hidden");
  window.scrollTo({ top: 0, behavior: "smooth" });
}
```

Génération d'un quiz par l'IA. La fonction `generateQuiz` lit les réglages, désactive les boutons pendant l'attente, envoie les paramètres au serveur avec `fetch` puis démarre le quiz avec les questions reçues. Les erreurs (sujet vide, serveur non lancé, quota dépassé…) sont affichées dans le message d'état :

```
async function generateQuiz() {
  const topic = topicInput.value.trim();
  if (!topic) {
    showSetupMessage("Saisissez un sujet pour générer les questions.", true)
   ;
    topicInput.focus();
    return;
  }

  generateButton.disabled = true;
  demoButton.disabled = true;
  showSetupMessage("Génération des questions en cours...");

  try {
    const response = await fetch("/api/questions", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        topic,
        count: Number(questionCountInput.value),
        type: questionTypeInput.value,
        difficulty: difficultyInput.value
      })
    });
    const data = JSON.parse(await response.text());
    if (!response.ok) throw new Error(data.error || "La génération a échoué.
   ");
    showSetupMessage("");
    startQuiz(data.questions);
  } catch (error) {
    showSetupMessage(error.message, true);
  } finally {
    generateButton.disabled = false;
    demoButton.disabled = false;
  }
}
```

Dans le code complet, la lecture de la réponse est protégée par un `try/catch` et un message explique à l'utilisateur de lancer le projet avec `npm start` lorsque le serveur n'est pas joignable (par exemple si la page est ouverte avec Live Server).

Démarrage et progression. La fonction `startQuiz` copie la série de questions, remet les compteurs à zéro et affiche la première question. À chaque question, `renderQuestion` met à

jour la barre de progression, le texte du bouton et rejoue l'animation d'arrivée :

```
function startQuiz(questionSet) {
  stopTimer();
  currentQuestions = questionSet.map((question) =>
    ({ ...question, answers: [...question.answers] }));
  currentQuestion = 0;
  score = 0;
  correctCount = 0;
  questionTotal.textContent =
    String(currentQuestions.length).padStart(2, "0");
  showScreen(quizScreen);
  renderQuestion();
  window.setTimeout(() => questionTitle.focus(), 50);
}

// dans renderQuestion() :
progressFill.style.width =
  `${((currentQuestion + 1) / currentQuestions.length) * 100}%`;
progressTrack.setAttribute("aria-valuenow", String(currentQuestion + 1));
nextButton.innerHTML = currentQuestion === currentQuestions.length - 1
  ? 'Voir mon résultat <span aria-hidden="true">→</span>'
  : 'Suivant <span aria-hidden="true">→</span>';
```

Génération des réponses (version simplifiée). À chaque question, les anciens boutons sont supprimés puis les nouveaux sont créés avec une lettre (A, B, C, D), le texte de la réponse et un marqueur ✓/× affiché après la correction. À la fin, le minuteur est relancé :

```
function renderQuestion() {
  const item = currentQuestions[currentQuestion];
  hasAnswered = false;
  questionTitle.textContent = item.question;
  questionKind.textContent =
    item.type === "true_false" ? "Vrai / Faux" : "QCM";
  answersContainer.replaceChildren();
  nextButton.disabled = true;

  item.answers.forEach((answer, index) => {
    const button = document.createElement("button");
    button.type = "button";
    button.className = "answer-option";
    button.setAttribute("aria-label",
      `Réponse ${String.fromCharCode(65 + index)} : ${answer}`);
    // ... lettre, texte et marqueur créés avec createElement
    button.addEventListener("click", () => selectAnswer(index));
    answersContainer.append(button);
  });

  startTimer();
}
```

Compte à rebours. Un `setInterval` recalcule toutes les 200 ms le temps restant à partir de l'heure de départ, ce qui évite toute dérive. Quand il atteint zéro, la question est validée automatiquement comme « hors délai » :

```
function startTimer() {
  stopTimer();
  questionStartedAt = Date.now();
  timeRemaining = QUESTION_TIME;
  updateTimerDisplay();

  timerId = window.setInterval(() => {
    timeRemaining = Math.max(0, Math.ceil(
      (QUESTION_TIME * 1000 - (Date.now() - questionStartedAt)) / 1000));
    updateTimerDisplay();
    if (timeRemaining === 0) selectAnswer(null, true);
  }, 200);
}

function updateTimerDisplay() {
  timerCountdown.textContent = `${timeRemaining} s`;
  timerTrack.setAttribute("aria-valuenow", String(timeRemaining));
  timerFill.style.width = `${(timeRemaining / QUESTION_TIME) * 100}%`;
  timerTrack.classList.toggle("timer-low", timeRemaining <= 5);
}
```

Correction et calcul du score. La fonction `selectAnswer` arrête le minuteur, vérifie la réponse, calcule les points en fonction du temps écoulé, colore les boutons et affiche l'explication. Le paramètre `timedOut` distingue le cas du temps écoulé :

```
function selectAnswer(selectedIndex, timedOut = false) {
  if (hasAnswered) return;
  hasAnswered = true;
  stopTimer();

  const item = currentQuestions[currentQuestion];
  const isCorrect = !timedOut && selectedIndex === item.correct;
  let pointsEarned = 0;
  if (isCorrect) {
    correctCount += 1;
    const elapsedSeconds =
      Math.floor((Date.now() - questionStartedAt) / 1000);
    pointsEarned = Math.max(10,
      POINTS_PER_QUESTION - elapsedSeconds * TIME_PENALTY_PER_SECOND);
    score += pointsEarned;
  }

  answersContainer.querySelectorAll(".answer-option")
    .forEach((button, index) => {
      button.disabled = true;
      if (index === item.correct) {
        button.classList.add("correct");
      } else if (!timedOut && index === selectedIndex) {
        button.classList.add("incorrect");
      }
    });

  feedback.className =
    `feedback ${isCorrect ? "correct-feedback" : "incorrect-feedback"}`;
  // titre : « Bonne réponse ! +N points », « Pas tout à fait ! »
```

```
  // ou « Temps écoulé ! », suivi de l'explication
  nextButton.disabled = false;
  nextButton.focus();
}
```

Meilleurs scores. Les cinq meilleurs scores sont stockés en JSON dans le `localStorage`. La lecture est protégée par un `try/catch` : si le stockage est indisponible ou corrompu, le quiz continue de fonctionner :

```
function getHighScores() {
  try {
    const saved = JSON.parse(localStorage.getItem(HIGH_SCORES_KEY) || "[]");

    return Array.isArray(saved)
      ? saved.filter((entry) => entry && Number.isFinite(entry.score)
          && Number.isFinite(entry.correct) && Number.isFinite(entry.total))

      : [];
  } catch {
    return [];
  }
}

function saveHighScore() {
  const scores = getHighScores();
  scores.push({ score, correct: correctCount,
    total: currentQuestions.length, date: new Date().toISOString() });
  scores.sort((a, b) => b.score - a.score || b.correct - a.correct);
  try {
    localStorage.setItem(HIGH_SCORES_KEY,
      JSON.stringify(scores.slice(0, 5)));
  } catch { /* le quiz fonctionne sans stockage */ }
  renderHighScores();
}
```

Affichage du résultat et événements. La fonction `showResults` enregistre le score, l'affiche et choisit un message selon le nombre de bonnes réponses ; les boutons sont reliés aux fonctions par des écouteurs d'événements. « Rejouer » relance la même série, « Nouveau quiz » revient à l'accueil :

```
function showResults() {
  stopTimer();
  saveHighScore();
  showScreen(resultScreen);
  scoreValue.textContent = String(score);
  scoreCaption.textContent =
    `${correctCount} / ${currentQuestions.length} bonnes réponses`;
  resultProgressFill.style.width =
    `${(correctCount / currentQuestions.length) * 100}%`;

  if (correctCount === currentQuestions.length) {
    resultTitle.textContent = "Incroyable !";
  } else if (correctCount >= Math.ceil(currentQuestions.length * 0.6)) {
    resultTitle.textContent = "Bien joué !";
```

```
  } else if (correctCount > 0) {
    resultTitle.textContent = "Joli début !";
  } else {
    resultTitle.textContent = "On recommence ?";
  }
}

generateButton.addEventListener("click", generateQuiz);
demoButton.addEventListener("click", () => startQuiz(demoQuestions));
restartButton.addEventListener("click", () => startQuiz(currentQuestions));
homeButton.addEventListener("click", returnToWelcome);
nextButton.addEventListener("click", () => {
  if (!hasAnswered) return;
  if (currentQuestion < currentQuestions.length - 1) {
    currentQuestion += 1;
    renderQuestion();
    questionTitle.focus();
  } else {
    showResults();
  }
});
```

Fonctionnement général. Le script gère trois écrans grâce à la fonction `showScreen`, qui masque tous les écrans puis affiche celui demandé. La fonction `generateQuiz` dialogue avec le serveur, `renderQuestion` construit l'interface de la question courante, `startTimer` gère le compte à rebours et `selectAnswer` traite le choix de l'utilisateur, ou l'expiration du temps, et calcule les points. La fonction `showResults` enregistre le score, l'affiche avec un message adapté (de « On recommence ? » à « Incroyable ! » selon le nombre de bonnes réponses) et met à jour la liste des meilleurs scores.

4.4 Exemple de code serveur (Node.js)

Le fichier `server.js` n'utilise que les modules intégrés de Node.js (`http`, `fs`, `path`) : aucune dépendance à installer. Il remplit trois rôles : servir les fichiers de la page, charger la clé d'API depuis `.env`, et fournir la route `POST /api/questions`.

Fichiers autorisés et routage. Seuls quatre chemins sont servis, ce qui empêche d'accéder à d'autres fichiers du dossier (notamment `.env`). Les requêtes `POST` vers `/api/questions` déclenchent la génération ; toute autre méthode reçoit une erreur 405 :

```
const allowedFiles = new Map([
  ["/", ["index.html", "text/html; charset=utf-8"]],
  ["/index.html", ["index.html", "text/html; charset=utf-8"]],
  ["/style.css", ["style.css", "text/css; charset=utf-8"]],
  ["/script.js", ["script.js", "text/javascript; charset=utf-8"]]
]);

const server = http.createServer(async (request, response) => {
  try {
    if (request.method === "POST" && request.url === "/api/questions") {
      await generateQuestions(request, response);
      return;
    }
```

```
    if (request.method === "GET") {
      serveFile(request.url, response);
      return;
    }
    sendJson(response, 405, { error: "Méthode non autorisée." });
  } catch (error) {
    sendJson(response, 500, { error: "Erreur interne du serveur. Réessayez.
    " });
  }
});

server.listen(PORT, "127.0.0.1");  // usage local uniquement
```

Validation des paramètres. Avant tout appel à l'API, le serveur vérifie que le sujet fait de 2 à 80 caractères, que le nombre de questions est un entier de 3 à 10 et que le type et le niveau font partie des valeurs autorisées. Le corps de la requête est limité à 10 000 caractères :

```
const { topic, count, type, difficulty } = settings || {};
const questionTypes = ["mixed", "multiple_choice", "true_false"];
const difficulties = ["easy", "medium", "hard"];

if (typeof topic !== "string" || topic.trim().length < 2
    || topic.length > 80) {
  sendJson(response, 400,
    { error: "Le sujet doit contenir entre 2 et 80 caractères." });
  return;
}
if (!Number.isInteger(count) || count < 3 || count > 10) {
  sendJson(response, 400, { error: "Choisissez de 3 à 10 questions." });
  return;
}
```

Appel à l'API Gemini. La clé est transmise dans l'en-tête `x-goog-api-key`. Un *schéma JSON* (`responseSchema`) impose à Gemini de répondre avec le nombre exact de questions et les champs attendus ; un délai maximal de 45 secondes évite d'attendre indéfiniment :

```
const model = process.env.GEMINI_MODEL || "gemini-3.5-flash-lite";
const apiResponse = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/${model}:
   generateContent`,
  {
    method: "POST",
    headers: {
      "x-goog-api-key": process.env.GEMINI_API_KEY,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      systemInstruction: { parts: [{ text: "Tu es un générateur de
        questions de quiz précis. ..." }] },
      contents: [{ role: "user", parts: [{ text: prompt }] }],
      generationConfig: {
        responseMimeType: "application/json",
        responseSchema: schema,   // count questions exactement
        maxOutputTokens: 3000
```

```
        }
      }),
      signal: AbortSignal.timeout(45000)
    }
);
```

Contrôle de la réponse. Même avec un schéma, la réponse d'un modèle d'IA n'est jamais prise pour acquise : la fonction `validateQuestions` vérifie le nombre de questions, le type, le nombre de réponses (2 pour vrai/faux, 4 pour un QCM) et la validité de l'indice de la bonne réponse, puis tronque les textes trop longs. Une réponse invalide produit une erreur plutôt qu'un quiz incohérent :

```
const expectedAnswers = question.type === "true_false" ? 2 : 4;
if (
  typeof question.question !== "string" ||
  !["multiple_choice", "true_false"].includes(question.type) ||
  (requestedType !== "mixed" && requestedType !== question.type) ||
  !Array.isArray(question.answers) ||
  question.answers.length !== expectedAnswers ||
  !Number.isInteger(question.correct_index) ||
  question.correct_index < 0 ||
  question.correct_index >= question.answers.length
) {
  throw new Error("The generated question was invalid.");
}
```

Gestion des erreurs. Les erreurs de l'API sont converties en messages clairs pour l'utilisateur, sans jamais divulguer la clé ni les détails internes : 429 (quota du niveau gratuit atteint), 503 (modèle surchargé), 502 (échec de génération) et 504 (délai dépassé).

Chargement de la clé. La fonction `loadLocalEnv` lit le fichier `.env` ligne par ligne et remplit `process.env`. Le fichier `.env` est listé dans `.gitignore` ; seul le modèle `.env.example` (avec une valeur fictive) fait partie du projet :

```
GEMINI_API_KEY=add-your-key-locally
GEMINI_MODEL=gemini-3.5-flash-lite
PORT=3000
```

5 Résultat final

5.1 Captures d'écran

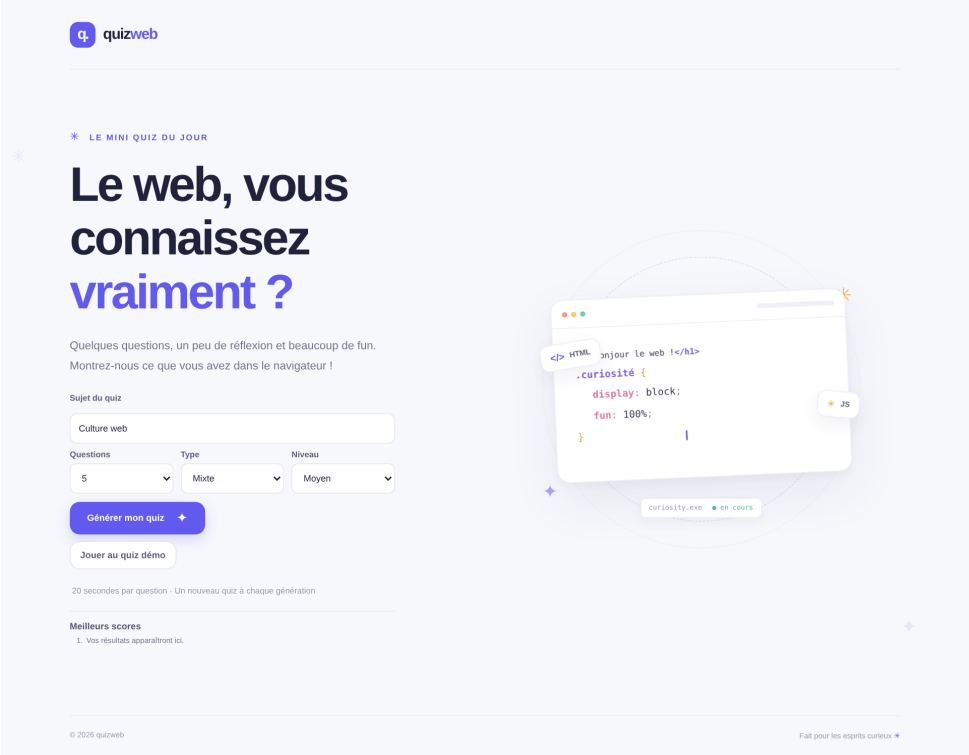


FIGURE 2 – Écran d'accueil du quiz : réglages, génération et quiz démo

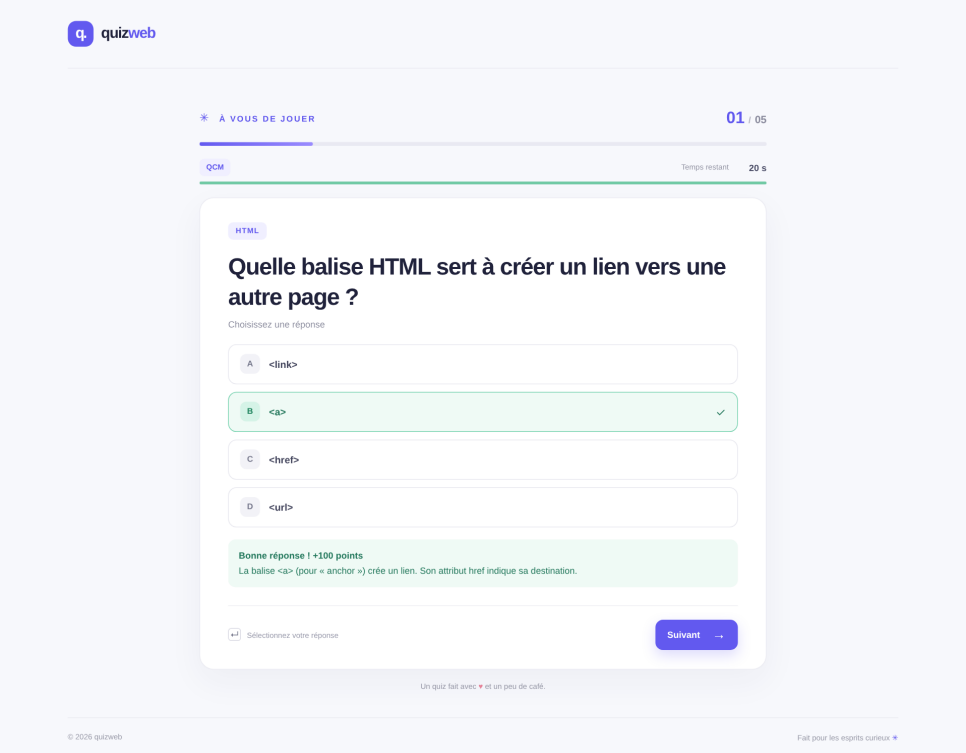


FIGURE 3 – Écran de question avec minuteur, correction et explication

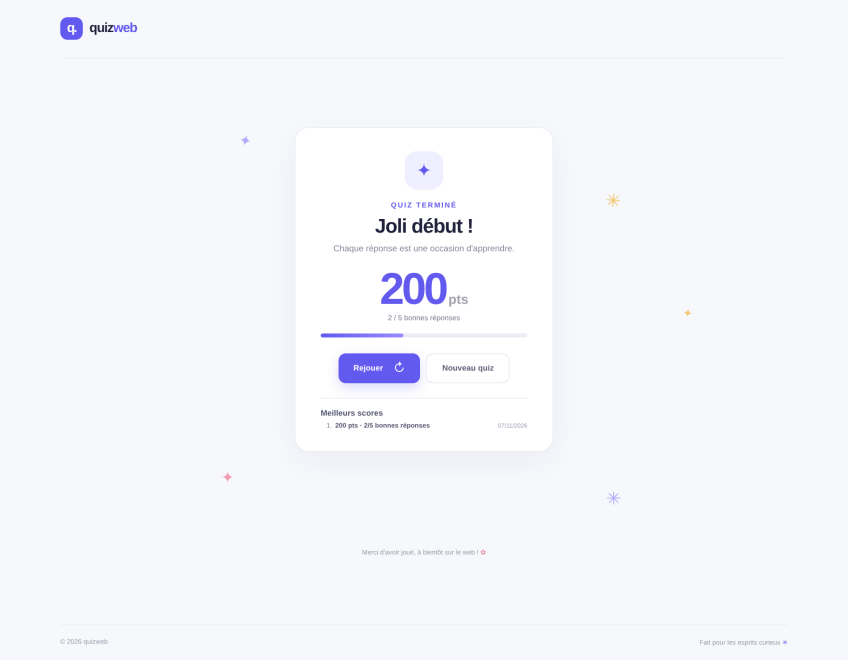


FIGURE 4 – Écran de résultat final avec score en points et meilleurs scores



FIGURE 5 – Page d'accueil sur un écran de téléphone (390 px de large)

5.2 Commentaire du rendu

— **Mise en page** : la disposition en deux colonnes de l'accueil (réglages et illustration) laisse place, pendant le quiz, à une carte centrée qui concentre l'attention sur la question ; le résultat est lui aussi présenté dans une carte centrée.
— **Retour visuel du temps** : la barre de temps se vide progressivement et devient rouge dans les cinq dernières secondes, ce qui crée une pression légère sans masquer la question.
— **Responsive design** : des media queries (à 760 px et 440 px) réorganisent la page sur une seule colonne, ajustent les marges et réduisent les tailles de police sur mobile.
— **Cohérence visuelle** : la même palette violette, les mêmes arrondis et les mêmes polices sont utilisés sur les trois écrans ; le vert et le rouge sont réservés au retour sur les réponses et au temps.
— **Robustesse** : des messages d'erreur explicites s'affichent sur l'accueil (sujet vide, serveur non lancé, quota dépassé) et le quiz démo reste toujours disponible, même sans clé d'API.
— **Accessibilité** : barres de progression avec rôle `progressbar`, annonces vocales via `aria-live`, focus déplacé sur la question et sur le bouton « Suivant », et respect de `prefers-reduced-motion`.

5.3 Performance

La performance d'une page web influence directement l'expérience de l'utilisateur : une page qui s'affiche vite et réagit sans délai donne une meilleure impression et reste utilisable sur une connexion lente ou un téléphone modeste. Plusieurs choix de conception ont été faits dans ce sens : aucune bibliothèque JavaScript ou CSS externe, un seul script chargé avec `defer`, aucune image (les illustrations sont réalisées en CSS) et un serveur qui renvoie directement les fichiers.

La page d'accueil a été auditée avec Lighthouse (Chrome, mode ordinateur) sur le serveur local. Les résultats sont les suivants :

Catégorie	Score
Performance	100
Accessibilité	96
Bonnes pratiques	96
SEO	100

TABLE 1 – Scores Lighthouse de la page d'accueil

Les principales métriques mesurées sont : First Contentful Paint 1,4 s, Largest Contentful Paint 1,4 s, Total Blocking Time 0 ms, Cumulative Layout Shift 0 et Speed Index 1,4 s.

Lighthouse signale deux points d'amélioration en accessibilité : le contraste de certains textes secondaires gris clair (texte d'aide, mentions en petits caractères), et le nom accessible du logo (`aria-label="Quiz Web, accueil"`) qui ne reprend pas exactement le texte visible « quizweb ». Pour les bonnes pratiques, il relève l'absence de `favicon.ico` (réponse 404 du serveur).

5.4 Validation W3C

Pour vérifier la conformité du code aux standards du Web, les fichiers `index.html` et `style.css` ont été contrôlés avec le **Nu Html Checker**, le moteur du validateur officiel du W3C (https://validator.w3.org), qui vérifie également le CSS (https://jigsaw.w3.org/css-validator/) :

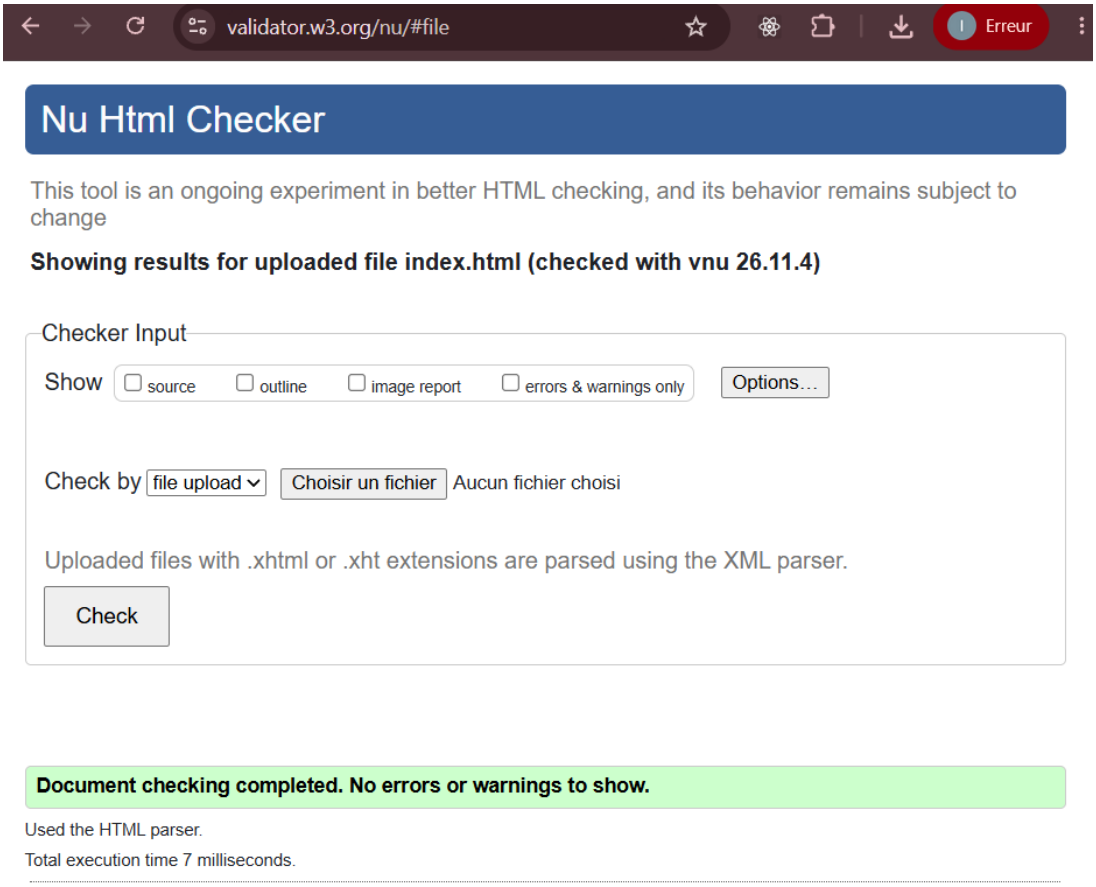


FIGURE 6 – Résultat du validateur W3C pour `index.html`

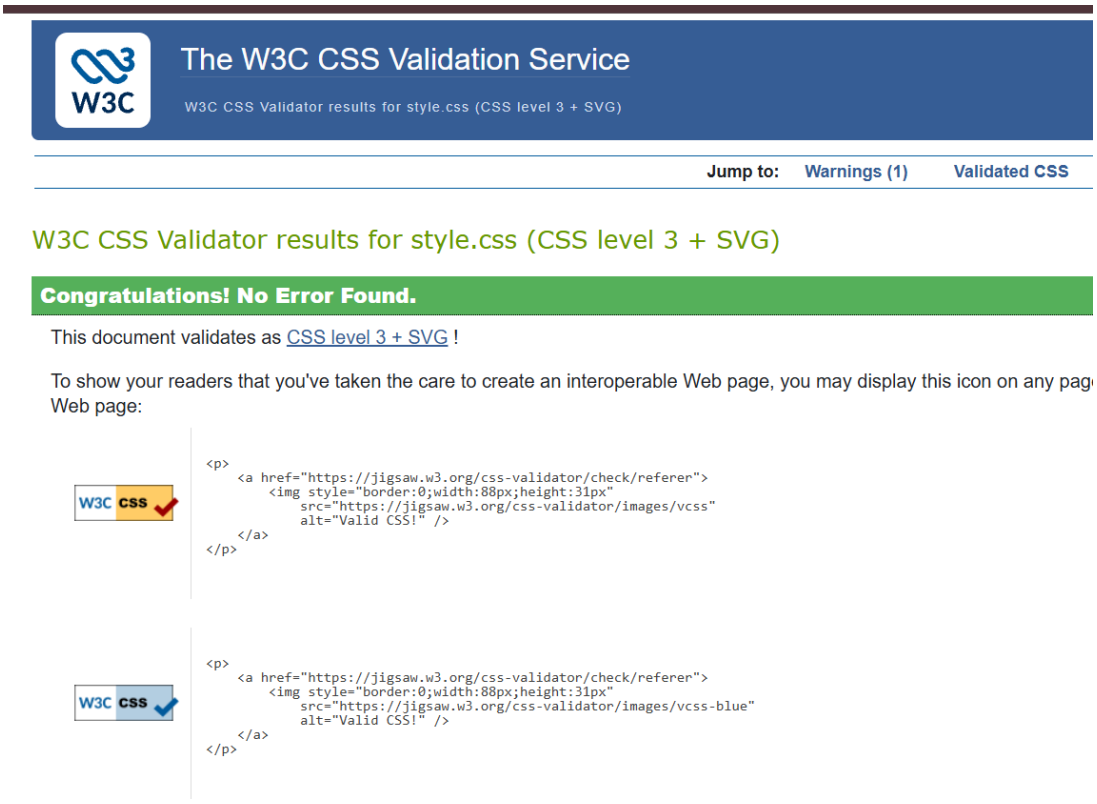


FIGURE 7 – Résultat du validateur W3C pour `style.css`

5.5 Vidéo de démonstration

Une vidéo d'enregistrement d'écran présente le fonctionnement complet de l'application, de l'écran d'accueil jusqu'au résultat final. Elle permet de voir en action les éléments qu'une capture d'écran ne peut pas montrer : le minuteur, les animations, la barre de progression et le retour immédiat après chaque réponse.

La vidéo est disponible en ligne : le lien ci-dessous (ou le QR code) permet de la visionner depuis n'importe quel lecteur PDF, sans rien télécharger.

Vidéo de démonstration :

Cliquer ici pour visionner la vidéo

Durée : 30 secondes – Format : MP4

Scanner pour voir la vidéo

Le fichier d'origine (`demo.mp4`) est également intégré à ce PDF : il est accessible depuis le panneau des pièces jointes du lecteur PDF (icône en forme de trombone).

Scénario présenté dans la vidéo.

1. l'écran d'accueil, les réglages du quiz et le bouton « Jouer au quiz démo » ;
2. le passage à la première question, avec le compteur, la barre de progression et le minuteur ;

3. le choix d'une mauvaise réponse : la bonne réponse est affichée en vert, le choix en rouge, accompagnés de l'explication ;
4. le choix de bonnes réponses, avec les points gagnés, puis le passage aux questions suivantes avec le bouton « Suivant » ;
5. l'écran de résultat avec le score en points, le message adapté et les meilleurs scores ;
6. le bouton « Nouveau quiz », qui ramène à l'accueil.

6 Difficultés rencontrées et solutions

— **Gestion de plusieurs écrans sur une seule page** : afficher l'accueil, le quiz et le résultat sans recharger la page. Solution : une classe utilitaire `.hidden` et une fonction `showScreen` qui bascule la visibilité des sections.
— **Génération dynamique des réponses** : éviter de dupliquer le HTML pour chaque question, d'autant que le nombre et le type des questions varient. Solution : création des boutons avec `document.createElement` et `replaceChildren` à chaque nouvelle question.
— **Multiples clics sur les réponses** : un utilisateur pouvait modifier son choix et fausser le score. Solution : un indicateur booléen `hasAnswered` et la désactivation des boutons après le premier clic.
— **Compte à rebours fiable** : un simple décrément à chaque tick dérive avec le temps. Solution : calculer le temps restant à partir de l'heure de départ (`Date.now()`) et toujours arrêter le minuteur (`stopTimer`) avant d'en lancer un nouveau, pour éviter plusieurs minuteurs simultanés.
— **Sécurité de la clé d'API** : une clé écrite dans `script.js` serait visible par tout visiteur. Solution : un petit serveur Node.js qui garde la clé dans `.env` (ignoré par Git), n'expose que quatre fichiers et écoute uniquement sur `127.0.0.1`.
— **Réponses imprévisibles de l'IA** : le modèle peut renvoyer un format incorrect. Solution : un schéma JSON imposé à Gemini, puis une validation stricte côté serveur (`validateQuestions`) avant d'envoyer les questions au navigateur.
— **Erreurs réseau et quotas** : l'API peut être surchargée, lente ou limitée. Solution : un délai maximal de 45 secondes, des codes d'erreur distincts (429, 503, 502, 504) et des messages clairs affichés sur l'accueil, avec le quiz démo comme solution de repli.
— **Problème d'alignement des éléments** : résolu grâce à `display: flex` et `display: grid`.
— **Mauvais rendu sur mobile** : corrigé avec des media queries adaptées.
— **Accessibilité** : gérer le focus et les annonces pour les lecteurs d'écran. Solution : appels à `focus()` aux moments clés, rôles et attributs ARIA, et annonces par `aria-live`.
— **Temps de chargement** : éviter que les polices et le script ne retardent l'affichage. Solution : attribut `defer` sur le script, `preconnect` pour les polices, aucune bibliothèque externe.

7 Conclusion

Ce projet m'a permis de consolider mes connaissances en HTML, CSS et JavaScript, notamment sur la structuration d'une page web, l'utilisation des sélecteurs et des variables CSS, la manipulation du DOM et la conception responsive. Il a également été l'occasion de prendre en compte l'accessibilité dès la conception de l'interface.

L'ajout de la génération de questions par l'API Gemini m'a fait découvrir la communication entre un navigateur et un serveur (`fetch`, routes, codes HTTP, JSON), la gestion des erreurs, la validation des données et, surtout, la protection d'un secret comme une clé d'API. Le compte à rebours, le calcul des points et la sauvegarde des meilleurs scores dans le `localStorage` ont enrichi l'expérience de jeu.

Le projet pourrait encore évoluer, par exemple avec des comptes utilisateurs et un classement partagé, un déploiement en ligne avec un hébergeur pour le serveur, ou l'export des questions générées.